Bridging the Enforcement Verification Gap: Detecting Divergence Between Declared and Actual Security Posture in Kubernetes Containerized Environments

Yehia Sobeh
Department of Cybersecurity
Email: yehiasobehacm@gmail.com

***Abstract*—Container orchestration platforms such as Kubernetes rely on declarative security primitives to enforce security boundaries around containerized workloads. A fundamental assumption underlies this model: that the container runtime faithfully translates declared security intent into kernel-level enforcement. This paper identifies and addresses the *enforcement verification gap*: the absence of tooling that systematically verifies whether declared Kubernetes security contexts are actually enforced by the container runtime. We present k8s-psa, a DaemonSet-based auditor that continuously compares pod security specifications retrieved from the Kubernetes API against the actual runtime state obtained through the container runtime interface. Through controlled experiments on a Minikube cluster with the Docker runtime, we evaluated the tool across seven test scenarios spanning fully hardened, partially secured, privileged, and deliberately misconfigured workloads. Our results reveal two categories of findings. First, privileged containers that declare seccomp system call filtering have that filtering silently stripped by the Docker runtime, a genuine enforcement gap that is invisible to existing specification-only tools. Second, the Docker runtime metadata interface omits the seccomp enforcement status of non-privileged containers that are correctly protected, an observability gap that produces false positives in naive runtime inspection. A kernel-verified cross-runtime validation on containerd shows that the enforcement gap generalizes, since containerd likewise strips declared seccomp profiles from privileged containers, whereas the observability gap is Docker-specific. These findings demonstrate that the enforcement verification gap is real, that it manifests differently across container configurations and runtimes, and that runtime inspection must account for the semantic differences between runtime metadata and actual kernel enforcement.**

***Index Terms*—Kubernetes, container security, seccomp, runtime verification, enforcement drift, DevSecOps, security context**

I. Introduction

Modern software systems increasingly adopt microservice architectures deployed as containerized workloads on Kubernetes. This shift introduces security challenges that traditional perimeter-based approaches cannot address: a larger attack surface distributed across potentially hundreds of containers, complex dependency chains that pull in third-party images, and dynamic runtime behavior in which containers are continuously created, scaled, and terminated [1], [2].

Kubernetes addresses these challenges through declarative security primitives specified in pod security contexts. Admission controllers such as OPA Gatekeeper [3] and Kyverno [4] intercept pod creation requests and reject non-compliant specifications, while Pod Security Admission (PSA), which has been stable since Kubernetes v1.25 [5], enforces predefined security profiles at the namespace level.

Despite their differences, these tools share a critical limitation: every one of them reads security properties from the Kubernetes API, that is, from the *declared* pod specification. None of them verifies that the container runtime has correctly translated those declarations into kernel-level enforcement. This verification matters because the translation from a Kubernetes pod specification to actual process isolation passes through multiple layers: the kubelet translates the pod specification into a Container Runtime Interface (CRI) request, the container runtime generates an Open Container Initiative (OCI) runtime specification, and a low-level runtime configures Linux kernel primitives [6]. Each translation step may add defaults, override values, or fail to apply certain restrictions.

This paper terms the resulting blind spot the *enforcement verification gap* and presents k8s-psa,[1] a tool designed to detect divergences between declared security intent and actual runtime enforcement.

A. Research Questions

- **RQ1:** To what extent does the actual security configuration of running containers diverge from the security properties declared in their Kubernetes pod specifications?
- **RQ2:** Can a lightweight, DaemonSet-based tool effectively detect such divergences with acceptable accuracy and performance overhead?
- **RQ3:** What categories of divergence occur in practice, and how should each category be classified from a security perspective?

[1]Kubernetes Pod Security Auditor; not to be confused with the built-in Pod Security Admission (PSA) controller, which operates only on declared specifications.

B. Contributions

This work makes four contributions: (1) a novel verification tool that closes the enforcement verification gap by systematically comparing declared Kubernetes security contexts against observed container runtime state; (2) experimental evidence of enforcement drift in privileged containers, where both Docker and containerd strip declared seccomp profiles without warning, confirmed at the kernel level via `/proc/<pid>/status`; (3) the discovery of an observability gap in the Docker runtime metadata interface, which causes correctly protected containers to appear unprotected and which does not occur on containerd; and (4) an open-source evaluation framework comprising 44 unit tests, 7 deployment-level test cases, and a real-time dashboard.

II. Background

A. Kubernetes Security Architecture

Kubernetes enforces workload security through a layered model. The Kubernetes API accepts pod specifications containing `securityContext` fields that define the intended security posture [6]. The kubelet translates these specifications into CRI requests, from which the container runtime generates an OCI runtime specification [7], a JSON document describing the exact Linux kernel configuration. A low-level runtime then configures namespaces, cgroups, capabilities, and seccomp filters.

B. Pod Security Admission and Policy Engines

Pod Security Admission, which has been stable since Kubernetes v1.25 [5], enforces Pod Security Standards at admission time but does not monitor running workloads or verify runtime enforcement. Policy engines extend this admission-time model with periodic re-evaluation: OPA Gatekeeper [3] audits cluster resources every 60 seconds, and Kyverno [4] performs background scans every hour. In both cases, however, the audit reads the declared pod specification from the Kubernetes API, and neither tool queries the container runtime.

C. Runtime Behavioral Monitoring

At the opposite end of the stack, behavioral monitors such as Falco [8] instrument the kernel via eBPF to detect anomalous system calls. These tools observe what containers *do* at the kernel level, but they do not verify whether the active security configuration matches what was declared.

III. Related Work

A. Static Misconfiguration Detection

Research on Kubernetes security configuration has concentrated on the specification level. Rahman et al. [11] conducted a large-scale empirical study of 2,039 open-source Kubernetes manifests and explicitly identified runtime verification as future work. Building on the same specification-level view, Malul et al. [12] proposed GenKubeSec, a fine-tuned language-model approach to YAML misconfiguration detection, and Sgan Cohen et al. [13] presented KubeGuard, which analyzes configuration files and runtime logs with a focus on RBAC and NetworkPolicy hardening; both works, which have not yet undergone peer review, likewise operate on declared configuration rather than enforced state.

B. Configuration Drift Detection

A second line of work targets configuration drift. Kubediff [14] reports differences between the running Kubernetes state and version-controlled manifests. The Kubernetes blog [15] described admission controllers for detecting container drift at runtime, and Komodor [16] documented operational approaches to drift detection. All of these approaches, however, treat the Kubernetes API as the source of truth for the running state; none extends below the API to the container runtime layer.

C. The Identified Gap

Several results indicate that the layer below the API deserves scrutiny. Kermabon-Bobinnec et al. [17] demonstrated that temporal inconsistency between the orchestration and runtime layers can be exploited to bypass security policies, and Shamim et al. [18] concluded that security researchers should investigate dynamic configuration violation detection. Survey work by Wong et al. [1] and Devi Priya et al. [2] likewise identified runtime configuration drift detection as under-explored. To the best of our knowledge, however, no existing open-source tool systematically compares the security declarations held by the Kubernetes API against the state reported by the container runtime. The present work addresses precisely this gap.

IV. Methodology

A. System Architecture

k8s-psa operates as a Kubernetes DaemonSet [10], deploying one scanner pod per cluster node. This deployment model guarantees complete node coverage, co-locates each scanner with the local container runtime socket, and scales automatically as nodes join or leave the cluster. Each instance performs periodic two-layer security scans; Fig. 1 shows the scanning pipeline.

Layer 1 (Spec Check) reads pod specifications from the Kubernetes API and checks them against eight configurable security policies (privileged mode, root user, host PID/network/IPC, seccomp profile, dangerous capabilities, and read-only root filesystem). This layer is functionally equivalent to existing policy engines and serves two purposes: it provides a baseline against which Layer 2 findings can be contextualized, and it flags pods that lack security declarations entirely, a case Layer 2 cannot detect, because the absence of a declaration leaves no intent to verify.

Layer 2 (Drift Detection), the novel contribution, compares declared specifications against actual runtime state by: (1) extracting a `SpecSecurityState` from the Kubernetes API; (2) querying the container runtime (`docker inspect` or `crictl inspect`) to build a `RuntimeSecurityState`;

(3) normalizing both states to account for expected divergences; and (4) comparing the normalized states field by field across seven security dimensions.

The comparison engine classifies each divergence as **CRITICAL** (a declared restriction is absent at runtime), **WARNING** (an undeclared capability is present), or **INFO** (the runtime is more restrictive than declared).

Reporting pipeline. Both layers emit their findings into a common reporting stage. An AlertManager consolidates the `spec_violations` produced by Layer 1 and the `drift_findings` produced by Layer 2, appends each record as one line of newline-delimited JSON to `findings.jsonl`, and exposes a lightweight dashboard over HTTP (port 8080) that renders the current findings for operators. This reporting stage forms the lower portion of Fig. 1.

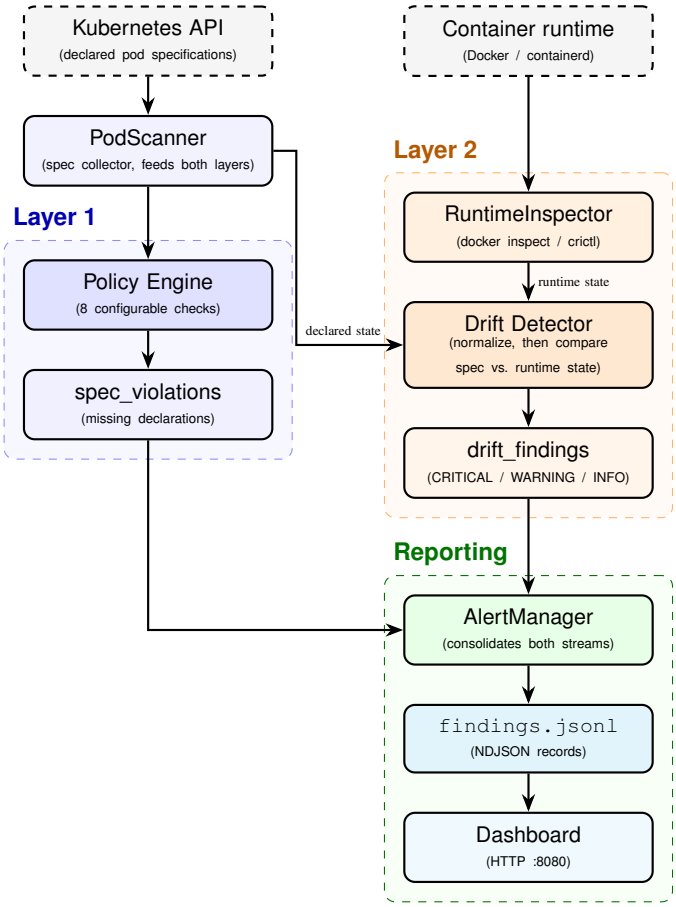


Fig. 1. Two-layer scanning pipeline of k8s-psa. The PodScanner is a shared spec collector: it reads declared specifications from the Kubernetes API and feeds both layers. Layer 1 validates those specifications against eight policies and reports missing declarations. Layer 2, the novel contribution, additionally obtains the actual runtime state through the RuntimeInspector, normalizes both states, and compares them field by field to detect divergence between declared and enforced security. The AlertManager then consolidates the two finding streams, appends them as NDJSON records to `findings.jsonl`, and the dashboard serves those records over HTTP for inspection.

B. Capability Normalization

Naive comparison of declared and observed capability sets would flood the operator with false positives, because the container runtime adds 14 default Linux capabilities to every non-privileged container [9]. The normalizer therefore computes the *expected* capability set from the pod specification: it starts with these documented defaults, removes the capabilities listed in `drop`, and adds those listed in `add`. Only deviations from this expected set generate findings.

C. Experimental Environment

Experiments were conducted on a single-node Minikube cluster (v1.35.0) running Kubernetes v1.32.0 with Docker 27.4.1. The host machine was a laptop with an Intel Core i5-1135G7 processor (4 cores, 8 threads, up to 4.2 GHz) and 8 GB of RAM, running Ubuntu 22.04.5 LTS with Linux kernel 6.8; the Minikube node was allocated 2 vCPUs and 2200 MB of memory through the Docker driver. The scanner interval was 60 seconds. Seven test pods were designed to cover the spectrum of security configurations: fully hardened, no security context, privileged with and without a declared seccomp profile, partially secured, one with privilege escalation restricted, and one sharing host namespaces. Three scan cycles were captured per pod. In addition to the seven purpose-built pods, the scanner observed the Kubernetes control plane pods running in the cluster, which serve as unplanned real-world subjects. Table I summarizes the hardware and software configuration.

TABLE I
EXPERIMENTAL SYSTEM SPECIFICATIONS

Component	Specification
Host CPU	Intel Core i5-1135G7 (4C/8T, up to 4.2 GHz)
Host memory	8 GB
Host OS	Ubuntu 22.04.5 LTS
Host kernel	Linux 6.8
Minikube	v1.35.0 (Docker driver)
Node resources	2 vCPUs, 2200 MB memory
Kubernetes	v1.32.0
Primary runtime	Docker 27.4.1
Cross-runtime	containerd 1.7.24, runc 1.2.2
Scan interval	60 s

To test whether the findings are Docker-specific, we additionally performed a targeted cross-runtime validation on a second cluster on the same host, with the identical Kubernetes version (v1.32.0) and resource allocation but the containerd runtime (containerd 1.7.24, runc 1.2.2), redeploying the decisive test pods (1, 3, and 4) and verifying enforcement both through `crictl inspect` and directly against the kernel via `/proc/<pid>/status`.

V. RESULTS

A. Detection Results Summary

Table II presents the Layer 2 drift findings for each test pod. Layer 1 behaved as expected across all seven pods (e.g., zero violations for the fully hardened Pod 1 and six violations for

the host-namespace Pod 7) and is not discussed further; the remainder of this section focuses on the Layer 2 findings that constitute the contribution of this work.

TABLE II
LAYER 2 DRIFT FINDINGS BY TEST POD

Test Pod	CRIT	WARN	INFO	Match
1. Fully Hardened	1*	0	0	§V-C
2. No Security Ctx	0	0	0	Yes
3. Priv + Seccomp	1	27	0	Yes
4. Priv, No Seccomp	0	27	0	Yes
5. Partial Security	1*	0	0	§V-C
6. No Privesc	1*	0	0	§V-C
7. Host Namespaces	0	0	2	Yes

*Observability false positives analyzed in Section V-C.

B. True Positive: Privileged Seccomp Stripping

The central finding concerns Test Pod 3, which declares both `privileged: true` and `seccompProfile.type: RuntimeDefault`. For such containers, the Docker runtime grants all 41 Linux capabilities and removes the seccomp filter entirely, even though seccomp filtering is explicitly declared. This behavior is documented in the Moby project issue tracker [19]. The tool reported the divergence as follows:

```
{
  "field": "seccompProfile",
  "severity": "critical",
  "category": "seccomp_mismatch",
  "description": "Spec declares seccomp type
    'RuntimeDefault' but the runtime has NO
    seccomp filter (unconfined)",
  "declared": "RuntimeDefault",
  "actual": "unconfined (null)"
}
```

Test Pod 4 serves as the experimental control: it has the same runtime state as Pod 3 but no seccomp declaration, and it produced zero CRITICAL findings. The 27 WARNING-level capability findings are identical between Pods 3 and 4, reflecting the 27 capabilities that privileged mode grants beyond the 14 defaults, including `CAP_SYS_ADMIN`, `CAP_NET_ADMIN`, and `CAP_SYS_MODULE`.

The security implication is direct. An operator who declares `RuntimeDefault` on a privileged container reasonably expects system call filtering to be active. Any specification-only tool reads the Kubernetes API, sees `RuntimeDefault`, and reports the container as filtered. Only by querying the container runtime does this removal become visible.

C. Observability Gap

Test Pods 1, 5, and 6, all non-privileged and all declaring `RuntimeDefault`, unexpectedly produced CRITICAL seccomp findings. Investigation revealed that these are false positives caused by a documented but widely misunderstood behavior of the Docker metadata interface. When cri-dockerd translates `RuntimeDefault`, it passes no explicit `SecurityOpt` entry to Docker and instead relies on Docker's implicit default profile [9], which restricts several dozen sensitive system calls such as `mount`, `ptrace`, and `reboot` [20]. Because the implicit profile never appears in `docker inspect` output, a correctly protected container is indistinguishable at the metadata level from an unprotected one. The CIS Docker Benchmark (check 5.21) documents exactly this semantics: an empty `SecurityOpt` field indicates that the default profile is *active*, not that seccomp is disabled [21], [22].

The Kubernetes control plane pods observed during the evaluation (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) exhibited the same phenomenon, each producing one false-positive CRITICAL seccomp finding. This consistency across purpose-built and real-world workloads confirms that the observability gap is systematic rather than an artifact of the test pod design.

D. Pod 3 vs. Pod 4: Controlled Comparison

Table III presents the controlled comparison between Pod 3 and Pod 4, which isolates the seccomp declaration as the single independent variable.

TABLE III
CONTROLLED COMPARISON: POD 3 VS. POD 4

Property	Pod 3	Pod 4
Spec: privileged	true	true
Spec: seccompProfile	RuntimeDefault	Not declared
Runtime: capabilities	All 41	All 41
Runtime: seccomp	Unconfined	Unconfined
L2 CRITICAL	**1**	**0**
L2 WARNING	27	27

Both pods have identical runtime states; the only difference is the seccomp declaration. The drift detector flags Pod 3 and remains silent on Pod 4, demonstrating its *specificity*: the tool distinguishes "restriction declared but not enforced" from "restriction never declared."

E. Cross-Runtime Validation on containerd

Table IV summarizes the validation experiment on the containerd cluster, with the kernel's own seccomp status as ground truth.

TABLE IV
CROSS-RUNTIME VALIDATION (CONTAINERD 1.7.24, RUNC 1.2.2)

Check	Pod 1	Pod 3	Pod 4
OCI spec: seccomp	Present	Absent	Absent
OCI spec: capabilities	0	41	41
Kernel: `Seccomp` field	**2** (filter)	**0** (none)	**0** (none)
Kernel: `CapBnd` bits	0	41	41

Two conclusions follow. First, *the enforcement gap generalizes beyond Docker*: Pod 3's declared `RuntimeDefault` profile is passed by the kubelet through the CRI but stripped by containerd, and the kernel confirms that no filter is installed (`Seccomp: 0`). The paper's central finding is therefore not a Docker quirk: it holds on the runtime that dominates production clusters. Second, *the observability gap is Docker-specific*:

`crictl inspect` on containerd exposes the full OCI runtime specification, in which Pod 1's seccomp allowlist is explicitly visible and matches the kernel state (`Seccomp: 2`). The false positives observed on Docker would not occur on containerd.

The validation also surfaced a further metadata-semantics trap: in the CRI protobuf JSON, a seccomp profile of type `RuntimeDefault` serializes as an empty object (enum value 0 is omitted), so naive inspection of the CRI configuration would misread a correctly declared profile as absent. This reinforces the paper's argument that verification tools must interpret runtime metadata semantics rather than take them at face value.

F. Performance Characteristics

On the experimental system detailed in Section IV-C and Table I (Intel Core i5-1135G7, 8 GB RAM, Minikube node limited to 2 vCPUs and 2200 MB), each scan cycle completed in approximately 3.1 seconds for 16 pods (0.7 s for Layer 1 and 2.4 s for Layer 2 across 18 containers), well within the 60-second scan interval. The scanner consumed 64 MiB of memory and fewer than 100m CPU, and the dashboard a further 32 MiB. The tool ships with 44 automated unit tests that execute in 0.2 seconds.

VI. Discussion

A. Answers to the Research Questions

RQ1 (extent of divergence). Divergence between declared and enforced security is not hypothetical. Privileged containers with seccomp declarations exhibit a real enforcement gap in which the runtime drops the declared restriction unannounced, on Docker and containerd alike, and privileged mode inflates the capability set by 27 capabilities beyond the declared expectation.

RQ2 (feasibility of lightweight detection). A DaemonSet-based tool can detect such divergences with modest resources (64 MiB, <100m CPU, 3.1 s per cycle). The controlled Pod 3/Pod 4 comparison shows that detection is specific rather than merely sensitive; the observability false positives show that accuracy depends on correctly interpreting runtime metadata semantics.

RQ3 (categories of divergence). Three categories emerged: (1) a *genuine enforcement gap* (CRITICAL), where a declared seccomp profile is stripped by privileged mode; (2) *capability inflation* (WARNING), where privileged mode grants all 41 capabilities; and (3) an *observability gap* (false-positive CRITICAL), where Docker's implicit seccomp default is invisible to metadata inspection.

B. The Gap Is Real but Nuanced

The results therefore confirm the enforcement verification gap while revealing that it has two distinct forms. The enforcement gap creates a false sense of security, because the declared control simply is not there, and the cross-runtime validation shows it is systemic rather than an implementation quirk of one runtime. The observability gap, by contrast, is a property of the Docker metadata interface, and it demonstrates that runtime metadata is a lossy abstraction over kernel state. A definitive verdict requires reading the kernel's own enforcement status from `/proc/<pid>/status`, where a `Seccomp` value of 2 confirms that BPF filter mode is active [23]; our validation experiment used exactly this check as ground truth.

C. Comparison with Existing Tools

Table V positions k8s-psa in the existing tool landscape. To the best of our knowledge, no existing open-source tool performs the spec-versus-runtime comparison that k8s-psa implements: OPA Gatekeeper [3] and Kyverno [4] read only the Kubernetes API, Falco [8] monitors kernel behavior without spec comparison, and Kubediff [14] reports differences between the API state and git manifests. k8s-psa occupies the space between specification-level tools and kernel-level monitors.

TABLE V
Positioning of k8s-psa Among Existing Tools

Tool	What It Checks	Layer
OPA Gatekeeper	Spec vs. policy constraints	API only
Kyverno	Spec vs. validate rules	API only
Falco	Syscall behavior (eBPF)	Kernel only
Kubediff	Spec vs. git manifests	API vs. git
k8s-psa	**Spec vs. runtime state**	**API vs. runtime**

D. Limitations

Five limitations bound the generality of these results. First, the full tool evaluation ran on Docker; the containerd validation covered the three decisive pods rather than the complete scenario matrix, and CRI-O remains untested. Second, the tool's automated seccomp verification relies on runtime metadata rather than kernel-level status (the kernel check in Section V-E was performed manually and is not yet integrated into the scanner). Third, the evaluation was conducted on single-node clusters. Fourth, the tool shells out to CLI commands rather than using native API client libraries. Fifth, seven test pods cover the primary security dimensions but not every configuration (e.g., init, sidecar, or ephemeral containers).

E. Future Work

Three directions merit investigation: integrating the `/proc/<pid>/status` kernel check, validated manually in this work, into the scanner itself, which would eliminate the observability gap entirely; extending the cross-runtime study to CRI-O and managed multi-node clusters (GKE, EKS) together with a prevalence study over real-world workloads; and investigating whether mutating admission webhooks introduce additional enforcement gaps by modifying security contexts after user submission.

VII. Conclusion

This paper identified and addressed the enforcement verification gap in Kubernetes security. Through controlled experiments, we demonstrated that privileged containers declaring

seccomp filtering had that filtering silently removed by the container runtime, on Docker and, as our kernel-verified cross-runtime validation shows, on containerd as well. This is a real enforcement failure that is invisible to all existing specification-only tools. We additionally discovered an observability gap in Docker's metadata interface that makes correctly protected containers appear unprotected; on containerd this gap does not occur. The tool, its evaluation framework, and its real-time dashboard are provided as open-source contributions.

References

[1] A. Y. Wong, E. G. Chekole, M. Ochoa, and J. Zhou, "On the security of containers: Threat modeling, attack analysis, and mitigation strategies," *Comput. Secur.*, vol. 128, art. 103140, May 2023, doi: 10.1016/j.cose.2023.103140.

[2] Devi Priya V S, S. C. Sethuraman, and M. K. Khan, "Container security: Precaution levels, mitigation strategies, and research perspectives," *Comput. Secur.*, vol. 135, art. 103490, Dec. 2023, doi: 10.1016/j.cose.2023.103490.

[3] Open Policy Agent, "Gatekeeper audit," 2024. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/audit/ (accessed Jul. 10, 2026).

[4] Kyverno, "Background scans," 2024. [Online]. Available: https://kyverno.io/docs/policy-reports/background/ (accessed Jul. 10, 2026).

[5] Kubernetes, "Kubernetes v1.25: Pod Security Admission controller in stable," Aug. 2022. [Online]. Available: https://kubernetes.io/blog/2022/08/25/pod-security-admission-stable/ (accessed Jul. 10, 2026).

[6] Kubernetes, "Restrict a container's syscalls with seccomp," 2024. [Online]. Available: https://kubernetes.io/docs/tutorials/security/seccomp/ (accessed Jul. 10, 2026).

[7] Open Container Initiative, "OCI runtime specification," GitHub, 2023. [Online]. Available: https://github.com/opencontainers/runtime-spec/blob/master/config.md (accessed Jul. 10, 2026).

[8] Sysdig, "Container drift detection with Falco," 2024. [Online]. Available: https://www.sysdig.com/blog/container-drift-detection-with-falco (accessed Jul. 10, 2026).

[9] Docker, "Seccomp security profiles for Docker," Docker Docs, 2024. [Online]. Available: https://docs.docker.com/engine/security/seccomp/ (accessed Jul. 10, 2026).

[10] Kubernetes, "DaemonSet," 2024. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/ (accessed Jul. 10, 2026).

[11] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, art. 99, pp. 1–36, May 2023, doi: 10.1145/3579639.

[12] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation," 2024, *arXiv:2405.19954*, doi: 10.48550/arXiv.2405.19954.

[13] O. Sgan Cohen, E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, "KubeGuard: LLM-assisted Kubernetes hardening via configuration files and runtime logs analysis," 2025, *arXiv:2509.04191*, doi: 10.48550/arXiv.2509.04191.

[14] Weaveworks, "Kubediff," GitHub, 2020. [Online]. Available: https://github.com/weaveworks/kubediff (accessed Jul. 10, 2026).

[15] Kubernetes, "Using admission controllers to detect container drift at runtime," Dec. 2021. [Online]. Available: https://kubernetes.io/blog/2021/12/21/admission-controllers-for-container-drift/ (accessed Jul. 10, 2026).

[16] Komodor, "Drift detection in Kubernetes," 2023. [Online]. Available: https://komodor.com/blog/drift-detection-in-kubernetes/ (accessed Jul. 10, 2026).

[17] H. Kermabon-Bobinnec *et al.*, "ProSPEC: Proactive security policy enforcement for containers," in *Proc. 12th ACM Conf. Data Appl. Security Privacy (CODASPY '22)*, Baltimore, MD, USA, Apr. 2022, pp. 1–12, doi: 10.1145/3508398.3511515.

[18] S. I. Shamim, H. Hu, and A. Rahman, "Dynamic application security testing for Kubernetes deployment: An experience report from industry," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng. (FSE '25), Industry Papers Track*, Trondheim, Norway, Jun. 2025, doi: 10.1145/3696630.3728573.

[19] Moby, "Allow for custom seccomp profiles to be used in privileged containers," GitHub, Issue #41321, 2020. [Online]. Available: https://github.com/moby/moby/issues/41321 (accessed Jul. 10, 2026).

[20] R. McCune, "Container security fundamentals part 6: Seccomp," Datadog Security Labs, Sep. 2023. [Online]. Available: https://securitylabs.datadoghq.com/articles/container-security-fundamentals-part-6/ (accessed Jul. 10, 2026).

[21] Center for Internet Security, "CIS Docker Benchmark," v1.7.0, 2024. [Online]. Available: https://www.cisecurity.org/benchmark/docker (accessed Jul. 10, 2026).

[22] Z. Mori, V. Daki, and T. avala, "Security hardening and compliance assessment of Kubernetes control plane and workloads," *J. Cybersecur. Priv.*, vol. 5, no. 2, art. 30, Jun. 2025, doi: 10.3390/jcp5020030.

[23] S. Grunert and M. Patel, "KEP-2413: Enable seccomp by default," Kubernetes Enhancement Proposal, 2023. [Online]. Available: https://github.com/kubernetes/enhancements/tree/master/keps/sig-node/2413-seccomp-by-default (accessed Jul. 10, 2026).